

Student Name	Sarvesh Naik
SRN No	31241943
Roll No	33
Program	Computer Engineering
Year	Third Year
Division	F
Subject	Artificial Intelligence Lab
Assignment No	1

Assignment 1

Problem Statement: *Implement Non-AI Techniques for the following problems:*

a. Tic Tac Toe b. N Queens c. Magic square

Objective:

To implement Non-AI problem solving techniques for classical problems such as Tic Tac Toe, N-Queens, and Magic Square, using systematic algorithms and logical approaches without applying artificial intelligence or heuristic-based methods.

The objective includes:

- To understand deterministic and rule-based problem solving techniques.
- To apply backtracking and logical validation methods for constraint satisfaction problems like N-Queens and Magic Square.
- To design a structured game logic for Tic Tac Toe using condition checking.
- To analyze how traditional algorithmic approaches differ from AI-based search techniques.
- To improve programming skills in implementing state validation, recursion, and combinatorial logic.

What are Non-AI Techniques:

Non-AI techniques are traditional problem-solving methods that use **predefined rules, logic, and deterministic algorithms** to solve problems.

These techniques:

- Do not learn from experience
- Do not use heuristics
- Do not make intelligent decisions
- Follow fixed instructions step-by-step

Non AI Techniques are rule based, deterministic, is has no learning abilities and it only works on Structured Approach like using loops, conditions, checks, recursion etc.

Non-AI techniques solve problems by:

1. Defining problem clearly
2. Applying mathematical or logical rules
3. Checking constraints
4. Using systematic search (if required)
5. Producing exact output

They rely on:

- Condition checking
- Backtracking
- Brute force methods
- Mathematical formulas

1. Tic Tac Toe

Tic Tac Toe is a simple two-player, turn-based game played on a 3×3 grid in which each player places their symbol, either X or O, in an empty cell during their turn. The main objective of the game is to align three identical symbols consecutively in a row, column, or diagonal before the opponent does.

In a non-AI implementation, the game operates using basic rule-based logic where the system checks for valid moves and verifies winning conditions after each turn through condition statements. However, when AI techniques are applied, the computer is designed to play intelligently against a human opponent. This is typically achieved using search algorithms such as the Minimax algorithm, which evaluate all possible future game states and choose the optimal move that maximizes the computer's chances of winning while minimizing the opponent's advantage. Unlike the non-AI approach, which simply follows predefined rules, the AI-based version involves decision-making, state evaluation, and strategic move selection, making the gameplay more challenging and intelligent.

Working:

- The game board is represented using a **linear data structure** with 9 positions.
- Players select a position from 1 to 9 to place their symbol.
- After each move, the program checks:
 - All rows
 - All columns
 - Both diagonals
- If any player forms a straight line of three symbols, that player wins.
- If all positions are filled and no winner is found, the game ends in a draw.

Pseudo Code:

```
1. Declare board as array of size 9
2. Initialize all board positions to EMPTY
3. Set currentPlayer = 'X'
4. DisplayBoard(board)
5. WHILE TRUE DO
    Print "Player ", currentPlayer, " turn"
    Input position
    IF position is valid AND board[position] is EMPTY THEN
        board[position] = currentPlayer
    ELSE
        Print "Invalid Move"
        CONTINUE
    ENDIF
    DisplayBoard(board)
    IF CheckWin(board, currentPlayer) = TRUE THEN
        Print "Player ", currentPlayer, " Wins!"
        BREAK
    ENDIF

    IF BoardFull(board) = TRUE THEN
        Print "Game Draw!"
        BREAK
    ENDIF

    IF currentPlayer = 'X' THEN
        currentPlayer = 'O'
    ELSE
```

```
        currentPlayer = 'X'

    ENDIF

END WHILE

END
```

Code:

```
# ===== TIC TAC TOE

# Draw the board

board = [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ']

def print_board():

    for i in range(0, 9, 3):

        print(board[i], "|", board[i+1], "|", board[i+2])

        if i < 6:

            print("---+---+---")

# Win check condition

def check_win(player):

    wins = [

        [0,1,2], [3,4,5], [6,7,8],

        [0,3,6], [1,4,7], [2,5,8],

        [0,4,8], [2,4,6]

    ]

    for w in wins:

        if board[w[0]] == player and board[w[1]] == player and board[w[2]] == player:
```

```
        return True

    return False

def is_draw():

    return ' ' not in board

current_player = 'X'

while True:

    print_board()

    pos = int(input(f"Player {current_player}, enter position (1-9): ")) - 1

    if board[pos] != ' ':

        print("Invalid move!")

        continue

    board[pos] = current_player

    if check_win(current_player):

        print_board()

        print(f"Player {current_player} wins!")

        break

    if is_draw():

        print_board()
```

```

print("Draw!")

break

current_player = 'O' if current_player == 'X' else 'X'

```

Output:

```

● sarveshmac@Sarveshs-MacBook-Air Artificial Intelligence % python3 Assignment-1.py
  |  |
--+--+
  |  |
--+--+
  |  |
Player X, enter position (1-9): 1
X |  |
--+--+
  |  |
--+--+
  |  |
Player O, enter position (1-9): 2
X | O |
--+--+
  |  |
--+--+
  |  |
Player X, enter position (1-9): 3
X | O | X
--+--+
  |  |
Player O, enter position (1-9): 4
X | O | X
--+--+
O |  |
--+--+
  |  |
Player X, enter position (1-9): 5
X | O | X
--+--+
O | X |
--+--+
  |  |
Player O, enter position (1-9): 6
X | O | X
--+--+
O | X | O
--+--+
  |  |
Player X, enter position (1-9): 7
X | O | X
--+--+
O | X | O
--+--+
X |  |
Player X wins!

```

2. N-Queens Problem

The N-Queens problem is a classical constraint satisfaction problem in which N queens are placed on an $N \times N$ chessboard such that no two queens attack each other, meaning no two queens share the same row, column, or diagonal. The objective is to arrange the queens so that all constraints are satisfied simultaneously.

In this implementation, a non-AI technique is used in the form of backtracking, which is a systematic and rule-based approach. The algorithm places queens one row at a time, checks for conflicts using logical condition statements, and if a conflict occurs, it backtracks to try another possible position. This method does not involve learning, heuristics, or intelligent decision-making; instead, it relies purely on deterministic logic, recursion, and constraint checking to find a valid solution.

Working:

- The solution uses the **backtracking technique**, which is a non-AI approach.
- Queens are placed one by one in different columns.
- For each placement, the program checks whether the position is safe.
- If a safe position is found, the queen is placed and the algorithm moves forward.
- If no safe position exists, the algorithm removes the previously placed queen and tries another position.
- This process continues until all queens are placed safely.

Pseudo Code:

```
FUNCTION SolveNQueens(board, col)

    IF col >= N THEN

        RETURN TRUE    // All queens placed successfully

    ENDIF

    FOR each row from 0 to N-1 DO

        IF IsSafe(board, row, col) = TRUE THEN

            board[row][col] = 1    // Place queen

            IF SolveNQueens(board, col + 1) = TRUE THEN

                RETURN TRUE

            ENDIF

            board[row][col] = 0    // Backtrack (remove queen)

        ENDIF

    END FOR

    RETURN FALSE    // No safe position found in this column

END FUNCTION
```

Code:

```

# ===== N QUEEN

# N Queens Problem using Backtracking

def solve_n_queens(n):

    board = [[0]*n for _ in range(n)]

    def is_safe(row, col):

        # Check row

        for i in range(col):

            if board[row][i] == 1:

                return False

        # Check upper left diagonal

        i, j = row, col

        while i >= 0 and j >= 0:

            if board[i][j] == 1:

                return False

            i -= 1

            j -= 1

        # Check lower left diagonal

        i, j = row, col

        while i < n and j >= 0:

            if board[i][j] == 1:

                return False

            i += 1

```

```

        j -= 1

    return True

def solve(col):

    if col >= n:

        return True

    for row in range(n):

        if is_safe(row, col):

            board[row][col] = 1

            if solve(col + 1):

                return True

            board[row][col] = 0

    return False

if solve(0):

    for row in board:

        print(row)

else:

    print("No solution")

solve_n_queens(4)

```

Output:

```
Enter the value of N: 4
[0, 0, 1, 0]
[1, 0, 0, 0]
[0, 0, 0, 1]
[0, 1, 0, 0]
```

3. Magic Square

A Magic Square is a square matrix of size $n \times n$ in which the numbers from 1 to n^2 are used exactly once, and the sum of every row, every column, and both main diagonals is the same. This common value is known as the Magic Constant. The objective is to arrange the numbers in such a way that all these conditions are satisfied simultaneously.

In this implementation, a non-AI technique is used by applying a predefined mathematical construction method called the Siamese Method (also known as De la Loubère's method) for odd-order magic squares. In this method, the number 1 is placed in the middle of the first row, and subsequent numbers are placed by moving one step upward and one step to the right. If this position is already occupied, the algorithm moves one step downward instead. The solution follows fixed rules and systematic logic without using learning, heuristics, or intelligent decision-making. Thus, the Magic Square is generated using deterministic mathematical principles rather than artificial intelligence techniques.

Working:

- The Magic Square is generated using the Siamese Method, which works only for odd values of n .
- The first number is placed in the middle of the top row.
- Each subsequent number is placed by moving:
 - One step upward
 - One step to the right
- If the calculated position lies outside the matrix, it wraps around.
- If the target cell is already occupied, the number is placed one position below the previous number
- This process continues until all values from 1 to n^2 are placed.

Pseudo Code:

START

Input n

IF n is even THEN

Print "Magic Square possible only for odd n "

STOP

ENDIF

Create matrix `magic[n][n]`

Initialize all elements to 0

Set row = 0

Set col = n // 2 // Middle column

FOR num = 1 to n*n DO

 magic[row][col] = num

 next_row = (row - 1 + n) % n

 next_col = (col + 1) % n

 IF magic[next_row][next_col] is NOT 0 THEN

 row = (row + 1) % n

 // col remains same

 ELSE

 row = next_row

 col = next_col

 ENDIF

END FOR

Display magic matrix

END

Code:

```
# ===== MAGIC SQUARE

# Magic Square for Odd n using Siamese Method

def magic_square(n):

    if n % 2 == 0:

        print("Magic square works only for odd n")

        return

    magic = [[0]*n for _ in range(n)]

    i = 0

    j = n // 2

    for num in range(1, n*n + 1):

        magic[i][j] = num

        new_i = (i - 1) % n

        new_j = (j + 1) % n

        if magic[new_i][new_j]:

            i = (i + 1) % n

        else:

            i = new_i

            j = new_j

    for row in magic:
```

```
print(row)

num = int(input("Enter the length square: "))

magic_square(num)
```

Output:

```
Enter the length square: 3
[8, 1, 6]
[3, 5, 7]
[4, 9, 2]
```